

Homework 01

680011465

2021-02-02T15:00:00 GMT

Contents

1. Read and Investigate the Data	2
2. Cluster Recipes	6
3. Investigate Clusters	11
4. Conclusion	13

1. Read and Investigate the Data

Read in the data.

```
recipes <- read.csv(file.choose())
```

```
View(recipes)
attach(recipes)
```

Make sure the ID variable is a character or factor, not numeric.

```
glimpse(recipes)
```

```
## Rows: 2,253
## Columns: 3
## $ recipe_id <int> 44902, 44902, 44902, 44902, 44902, 44902, 44902, 44902, ...
## $ ingredient <chr> "sugar", "large eggs", "all-purpose flour", "baking soda...
## $ cuisine    <chr> "southern_us", "southern_us", "southern_us", "southern_u...
```

```
recipes$recipe_id <- as.character(recipes$recipe_id)
class(recipes$recipe_id)
```

```
## [1] "character"
```

Initially, the data set shows that it is an ‘integer’. After using the ‘as.character’ command, it changes the ‘recipe_id’ to ‘character’

What variables are in the dataset? What data type are they?

```
glimpse(recipes)
```

```
## Rows: 2,253
## Columns: 3
## $ recipe_id <chr> "44902", "44902", "44902", "44902", "44902", "44902", "4...
## $ ingredient <chr> "sugar", "large eggs", "all-purpose flour", "baking soda...
## $ cuisine    <chr> "southern_us", "southern_us", "southern_us", "southern_u...
```

There are 3 variables. They are the recipe id, ingredients and cuisine. All of the 3 variables are character.

How many different unique ingredients are there?

```
n_distinct(ingredient)
```

```
## [1] 722
```

According to this command, there are 722 unique ingredients.

What are the top five most frequent ingredients?

```
recipes %>%
  count(ingredient) %>%
  top_n(5) %>%
  arrange(desc(n))
```

```
## Selecting by n
```

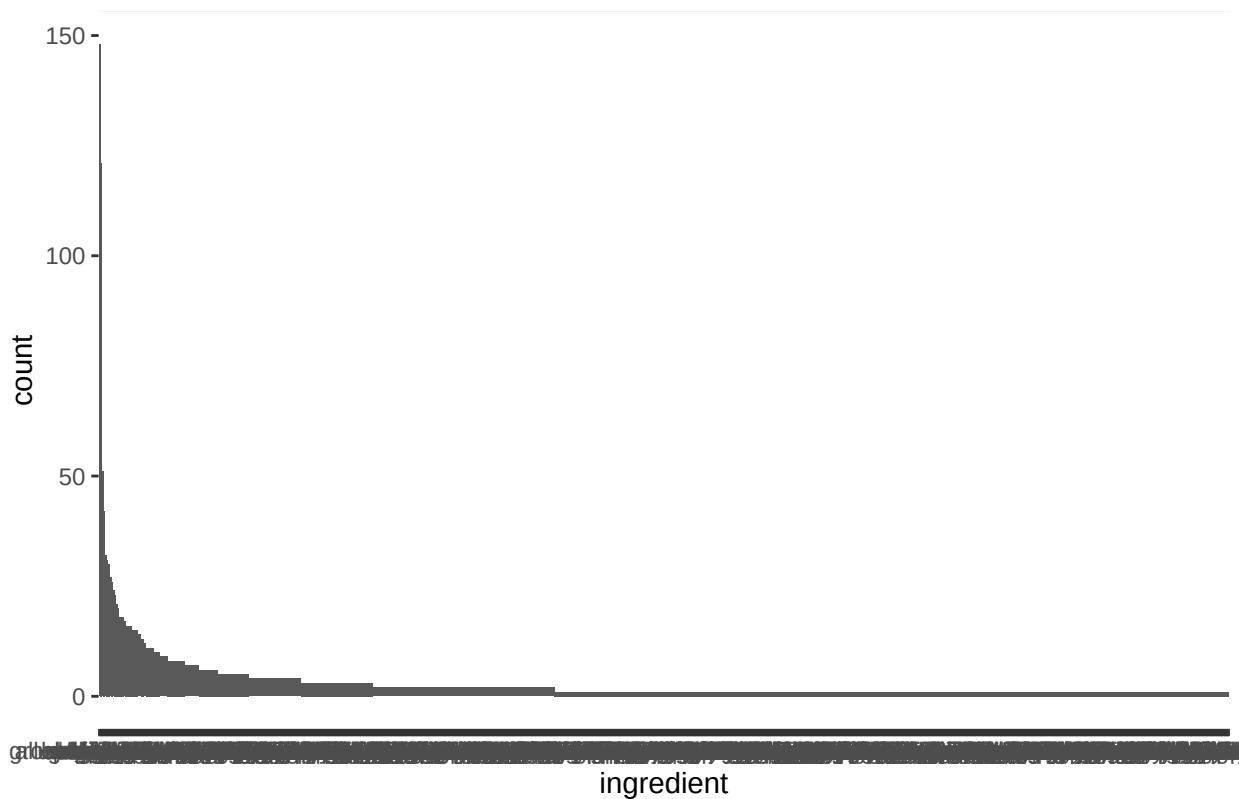
```
##      ingredient    n
## 1         salt 148
## 2         sugar 121
## 3         water  51
## 4 all-purpose flour 42
## 5         garlic 32
```

The top 5 ingredients are salt, sugar, water, all-purpose flour, garlic

Create a frequency bar chart of the ingredients showing the distribution of ingredients.

```
recipes %>%
  mutate(ingredient = fct_infreq(ingredient)) %>%
  ggplot() +
  geom_bar(mapping = aes(x = ingredient))+
  labs(title = "Distribution of ingredients")+
  theme(plot.title = element_text(face = "bold"))
```

Distribution of ingredients



```
scale_x_discrete(labels = NULL)
```

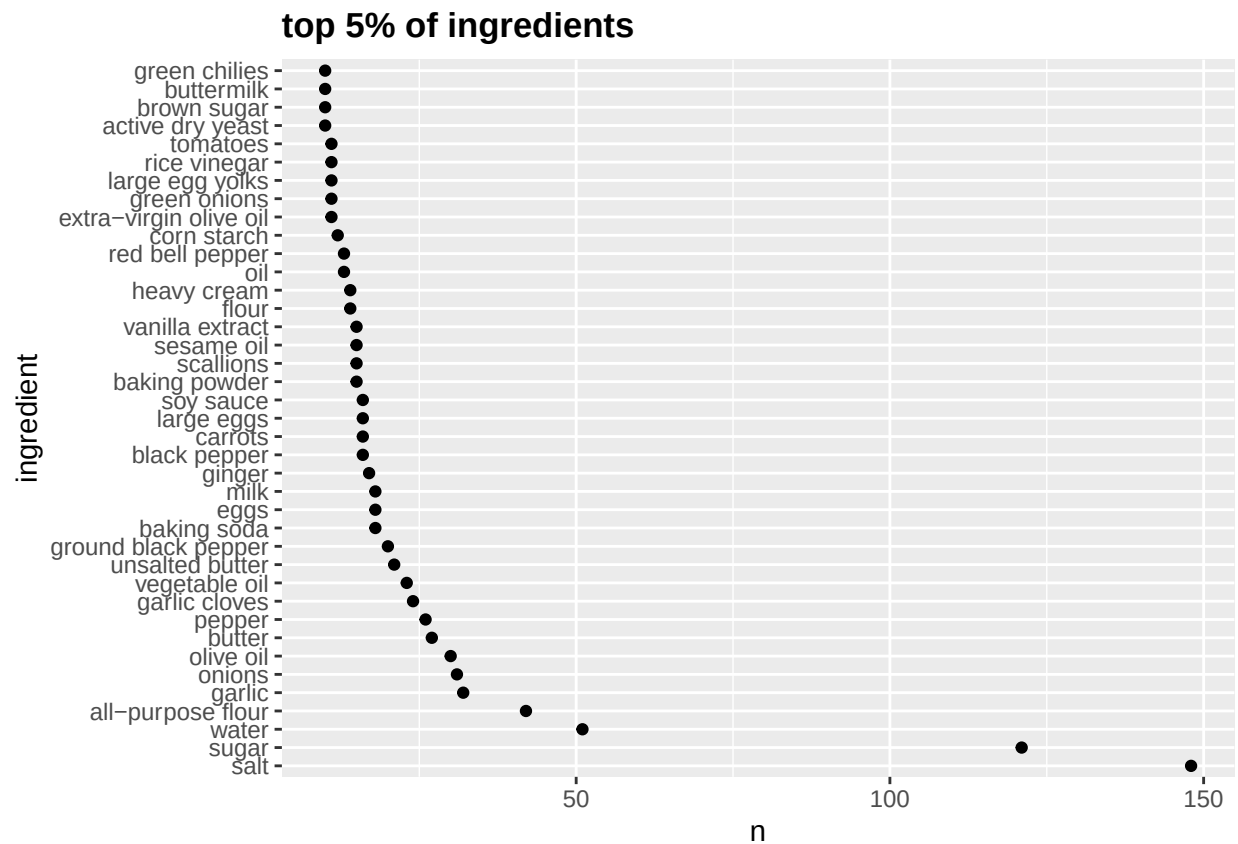
```
## <ggproto object: Class ScaleDiscretePosition, ScaleDiscrete, Scale, gg>
##   aesthetics: x xmin xmax xend
##   axis_order: function
##   break_info: function
##   break_positions: function
##   breaks: waiver
##   call: call
##   clone: function
##   dimension: function
##   drop: TRUE
##   expand: waiver
##   get_breaks: function
##   get_breaks_minor: function
##   get_labels: function
##   get_limits: function
##   guide: waiver
##   is_discrete: function
##   is_empty: function
##   labels: NULL
##   limits: NULL
##   make_sec_title: function
##   make_title: function
##   map: function
##   map_df: function
##   n.breaks.cache: NULL
##   na.translate: TRUE
##   na.value: NA
##   name: waiver
##   palette: function
##   palette.cache: NULL
##   position: bottom
##   range: <ggproto object: Class RangeDiscrete, Range, gg>
##     range: NULL
##     reset: function
##     train: function
##     super: <ggproto object: Class RangeDiscrete, Range, gg>
##   range_c: <ggproto object: Class RangeContinuous, Range, gg>
##     range: NULL
##     reset: function
##     train: function
##     super: <ggproto object: Class RangeContinuous, Range, gg>
##   rescale: function
##   reset: function
##   scale_name: position_d
##   train: function
##   train_df: function
##   transform: function
##   transform_df: function
##   super: <ggproto object: Class ScaleDiscretePosition, ScaleDiscrete, Scale, gg>
```

Optional: Create a dot plot of the top 5% of ingredients.

```

recipes %>%
  mutate(ingredient = fct_infreq(ingredient)) %>%
  count(ingredient) %>%
  top_frac(0.05, n) %>%
  ggplot() +
  geom_point(mapping = aes (x = n, y = ingredient)) +
  labs(title = "top 5% of ingredients")+
  theme(plot.title = element_text(face = "bold"))

```



2. Cluster Recipes

Uses this Jaccard similarity function to compare two sets of ingredients for a pair of recipes.

```
source('jaccard_matrix.R')
```

```
# m <- jaccard_matrix_fast(d = recipes,  
#                           unit_var = 'recipe_id',  
#                           item_var = 'ingredient',  
#                           distance = T)  
# saveRDS(m, file = "jaccard_distance_recipes.rds")  
m <- readRDS("jaccard_distance_recipes.rds")
```

Use `kmeans` to cluster the recipes. First determine the appropriate number of centroids to use. Compute the total within sum of squares using between one and twenty clusters. Then plot the results. Use the elbow method to determine number of clusters.

```
kmeans(m, 3)
```

```
## K-means clustering with 3 clusters of sizes 57, 61, 82  
##  
## Cluster means:  
##      10028      10290      10363      10381      10464      10568      10956  
## 1 0.9919442 0.9381722 0.9208160 0.9795739 0.9690430 0.9950727 0.9047016  
## 2 0.9386723 0.8751619 0.8345862 0.9494713 0.9244723 0.9438107 0.8301624  
## 3 0.9084975 0.9236015 0.9384687 0.9158893 0.9162669 0.9139723 0.9215408  
##      11425      11669      11837      11999      12414      12575      13139  
## 1 0.9944348 0.9948214 0.9164398 0.9827474 0.9951540 0.9713784 0.9899393  
## 2 0.9457863 0.9359691 0.9429465 0.9276485 0.9381959 0.9360794 0.9494530  
## 3 0.9302233 0.9283242 0.9832523 0.8970031 0.9160527 0.9138725 0.9140751  
##      13480      13987      14000      14276      14525      14566      14602  
## 1 0.9850616 0.9463656 0.9147988 0.9252204 0.9817397 0.9804735 0.8787683  
## 2 0.9417801 0.9028403 0.8379739 0.8319114 0.9389261 0.9303949 0.8815316  
## 3 0.8977673 0.9010634 0.9177788 0.9278464 0.9172498 0.9056139 0.9817904  
##      15077      15246      15646      15661      16021      16319      16394  
## 1 0.9307547 0.9952315 0.8949810 0.9925152 0.8961328 0.9073982 0.8914434  
## 2 0.8692335 0.9429461 0.9122788 0.9347675 0.9181583 0.9451174 0.9219787  
## 3 0.9442877 0.9269023 0.9301100 0.9192211 0.9878601 0.9735690 0.9850519  
##      1674      16940      17085      17166      17212      17344      17701  
## 1 0.9891961 0.9084504 0.9020441 0.9224292 0.9124197 0.9128283 0.9770788  
## 2 0.9147555 0.8048923 0.9333461 0.8516066 0.8494698 0.9129071 0.8754787  
## 3 0.8917802 0.9296597 0.9844173 0.9335061 0.9219063 0.9892050 0.9442020  
##      18005      18156      18435      1911      19176      19435      19644  
## 1 0.9738892 0.9841487 0.8944047 0.9305118 0.9122634 0.9838781 0.8939055  
## 2 0.9206684 0.9366618 0.9242132 0.9042485 0.9365546 0.9206766 0.9011889  
## 3 0.9088778 0.9054650 0.9908994 0.9157107 0.9614111 0.9350999 0.9353449  
##      19765      20011      20057      20078      20489      21058      21297  
## 1 0.9805485 0.9006319 0.9060960 0.9856934 0.9703744 0.9103798 0.8958796  
## 2 0.8532990 0.9342899 0.8967354 0.9431938 0.8733382 0.7854822 0.9526357  
## 3 0.9321803 0.9918979 0.9749071 0.9177139 0.9528510 0.9318737 0.9769262  
##      21323      21418      21557      21575      21673      22137      22904  
## 1 0.9400311 0.9811508 0.9767249 0.9863668 0.9870396 0.8823615 0.9244352
```

##	2	0.9015373	0.9478630	0.9446036	0.9297062	0.9137741	0.9320340	0.9513286
##	3	0.8963174	0.9247467	0.9149154	0.8982033	0.9262441	0.9805392	0.9924562
##		22930	23041	23062	23378	23664	23735	23974
##	1	0.9228537	0.8864005	0.9810253	0.9039355	0.9056668	0.9268899	0.9306821
##	2	0.8301244	0.9037076	0.9232853	0.8568029	0.7909250	0.8626074	0.8696768
##	3	0.9363619	0.9867807	0.8960271	0.9022221	0.9250308	0.9359220	0.9435937
##		2417	24185	24350	24826	25090	25453	25561
##	1	0.9403979	0.9056849	0.9359168	0.9181813	0.8933840	0.9451646	0.9186492
##	2	0.9217226	0.8243896	0.9032367	0.8141529	0.9436912	0.9122640	0.8377397
##	3	0.9158486	0.9234723	0.9159633	0.9352898	0.9588663	0.9257356	0.9346145
##		2572	26028	26091	26252	26290	26351	26513
##	1	0.8790958	0.8856820	0.9725603	0.9700670	0.9042526	0.9826525	0.9094832
##	2	0.8985729	0.8780122	0.9232108	0.8627357	0.9440734	0.9444684	0.8274335
##	3	0.9721214	0.9833806	0.9098972	0.9408796	0.9816508	0.9225448	0.9208977
##		26845	26989	2702	27122	27243	27577	27810
##	1	0.9752475	0.9138102	0.9659714	0.9693275	0.9130012	0.9170975	0.9141217
##	2	0.9282881	0.8740364	0.9046949	0.9192631	0.8663161	0.8473101	0.8022537
##	3	0.8847725	0.9161560	0.9193900	0.9217930	0.9317547	0.9341788	0.9293764
##		28378	28787	28886	29530	29685	2983	29846
##	1	0.9254510	0.9117694	0.9424294	0.8886188	0.9501945	0.9309071	0.9161868
##	2	0.8315220	0.9185042	0.9015684	0.9035868	0.9360615	0.8751704	0.9440760
##	3	0.9434323	0.9835669	0.8984679	0.9889451	0.9071949	0.9214443	0.9831087
##		29878	30191	30547	30728	30768	30782	30913
##	1	0.9681740	0.8952903	0.9229583	0.9943008	0.9090758	0.9249308	0.9702743
##	2	0.9067950	0.9425829	0.8103090	0.9180854	0.8266115	0.9246899	0.9445713
##	3	0.9109719	0.9578833	0.9368219	0.9087128	0.9293555	0.9910311	0.9206977
##		31381	31869	31872	32311	32397	33397	33579
##	1	0.9911985	0.9834010	0.9820285	0.8970735	0.9414959	0.9468028	0.9833577
##	2	0.9279627	0.9175661	0.9514941	0.9110372	0.9080777	0.9055907	0.9192864
##	3	0.8996854	0.9176763	0.9236898	0.9904434	0.9283520	0.9071838	0.9317170
##		33843	33933	34018	34987	35177	35314	35652
##	1	0.9188318	0.9812073	0.9814760	0.9900333	0.9951376	0.9686826	0.9940507
##	2	0.8665380	0.9506613	0.9500987	0.9326575	0.9188324	0.8418944	0.9252066
##	3	0.9383237	0.9318234	0.9390025	0.9018848	0.9140232	0.9262567	0.9155078
##		35771	35844	35998	36042	36181	36284	36519
##	1	0.9895274	0.9326020	0.9816984	0.9677467	0.9243876	0.9260911	0.9176275
##	2	0.9123082	0.8570036	0.9295600	0.8659549	0.9623818	0.8749523	0.9388645
##	3	0.8973538	0.9090689	0.8845437	0.9314142	0.9811409	0.9359539	0.9923311
##		36781	36792	37146	37628	3849	39095	39190
##	1	0.9799132	0.9056502	0.8811823	0.9330684	0.9142665	0.9784313	0.9887973
##	2	0.9387601	0.9290314	0.9289415	0.8600506	0.8107112	0.9302888	0.9311997
##	3	0.9047258	0.9901156	0.9690099	0.9392220	0.9357561	0.9157014	0.9126740
##		39541	39600	40120	40177	40200	40435	40610
##	1	0.8948170	0.9864273	0.8982107	0.9051630	0.9734087	0.9235621	0.9695126
##	2	0.9260842	0.9570397	0.9411486	0.9472370	0.9542176	0.8507380	0.8697710
##	3	0.9857909	0.9375954	0.9647795	0.9619955	0.9062116	0.9296857	0.9287977
##		40755	40817	4104	41290	41510	42079	42240
##	1	0.9362425	0.9443335	0.9913171	0.9827861	0.9211032	0.9749895	0.9543344
##	2	0.8788974	0.9352126	0.9375757	0.9359250	0.9344404	0.9493654	0.9240664
##	3	0.9474086	0.9268407	0.9245465	0.9017090	0.9910476	0.9067102	0.9325439
##		4226	42269	42356	4266	42782	42925	44027
##	1	0.8797067	0.8910272	0.9112288	0.9763356	0.9052229	0.9343560	0.9187547
##	2	0.9115515	0.7804379	0.8230236	0.9395510	0.9322311	0.8505891	0.8458056
##	3	0.9791051	0.9153275	0.9386019	0.9202778	0.9865695	0.9420235	0.9375911

```

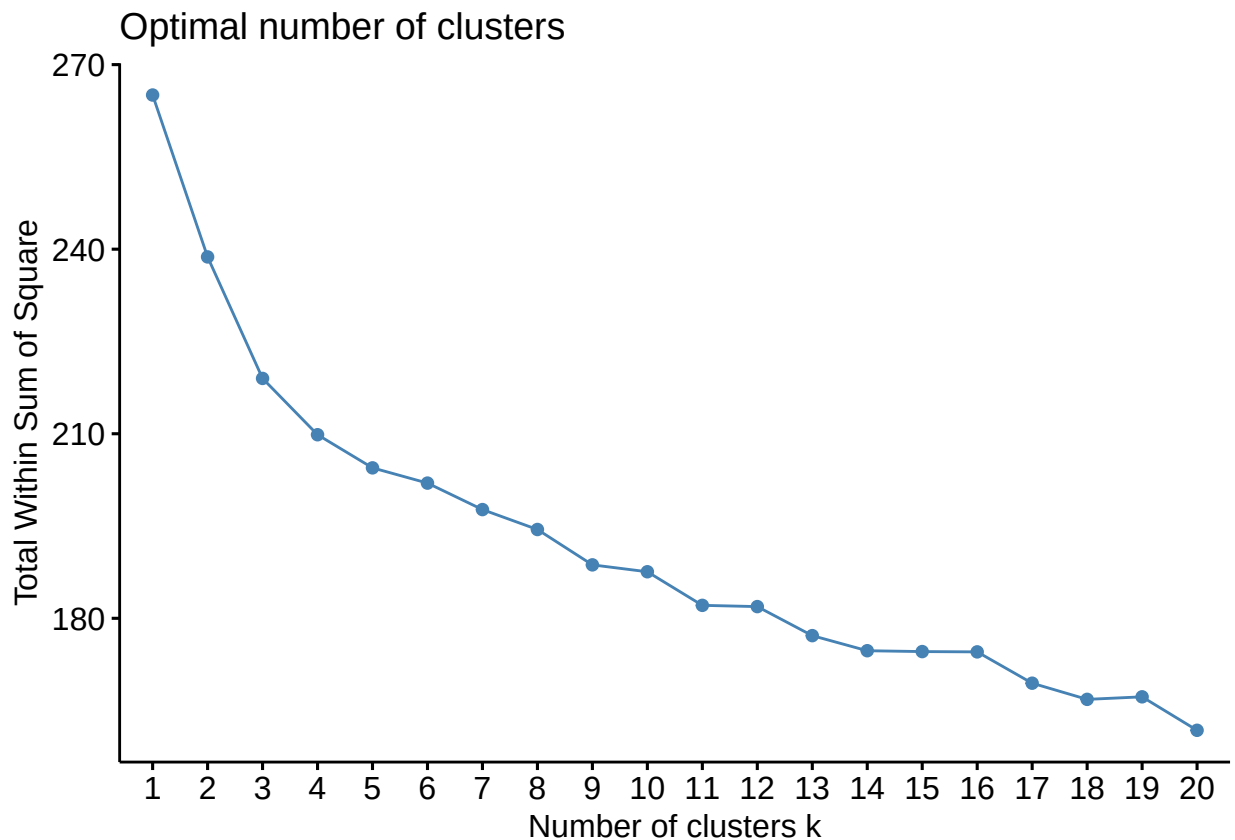
##      44279      44319      44327      44420      44826      44902      45207
## 1 0.8873100 0.8781322 0.9196385 0.9237154 0.9286029 0.9203464 0.9276026
## 2 0.9208557 0.9066742 0.8642058 0.9523453 0.8647283 0.7993628 0.9548160
## 3 0.9861799 0.9583383 0.9402146 0.9926100 0.9290917 0.9362542 0.9579070
##      45462      45617      4615      46354      46568      46609      46635
## 1 0.8963778 0.9768087 0.9928337 0.8975415 0.9086044 0.9496235 0.9828981
## 2 0.9142835 0.9344352 0.9387190 0.9190827 0.9185338 0.9204585 0.9407318
## 3 0.9842099 0.8976589 0.9131950 0.9863829 0.9632517 0.9136191 0.9110312
##      46646      46714      470      47045      47244      47421      478
## 1 0.9136178 0.9031014 0.9803827 0.9775357 0.9699397 0.9136243 0.9263664
## 2 0.9564094 0.8212529 0.9381372 0.9373050 0.8528343 0.9332871 0.8366856
## 3 0.9747327 0.9195621 0.9084431 0.9138710 0.9274104 0.9909066 0.9446675
##      48067      48251      48360      4842      48576      48997      49084
## 1 0.9246503 0.9418029 0.9107596 0.9225479 0.9278957 0.9652999 0.9949815
## 2 0.8527302 0.8777083 0.9493765 0.8698620 0.8732503 0.8979599 0.9528683
## 3 0.9134408 0.9206100 0.9866598 0.9305062 0.9122751 0.8852349 0.9293548
##      49328      49553      49630      5001      5808      6533      6954
## 1 0.9061461 0.9106682 0.9242167 0.9254996 0.9284371 0.9018250 0.8892235
## 2 0.9002217 0.9380089 0.9512009 0.8614309 0.8709910 0.9080361 0.9202047
## 3 0.9407928 0.9922139 0.9766678 0.9406035 0.9135144 0.9811395 0.9613624
##      7438      7601      8061      8115      8117      8684      8824
## 1 0.9824717 0.9814652 0.8960721 0.964372 0.9147029 0.9823845 0.9397642
## 2 0.9149433 0.9265117 0.9020553 0.926161 0.9475070 0.9127410 0.8238151
## 3 0.9187027 0.9047959 0.9330345 0.922718 0.9939112 0.9172179 0.9092875
##      8843      890      9048      9067      9202      9231      9253
## 1 0.9682950 0.9827608 0.9676679 0.9823250 0.8939973 0.9884009 0.9052445
## 2 0.9484935 0.9361361 0.9080824 0.9446283 0.9326896 0.9380534 0.8160477
## 3 0.9377507 0.8996885 0.9360438 0.8934796 0.9823002 0.8999748 0.9335269
##      9260      9727      9843      9882
## 1 0.9304590 0.8815523 0.9086262 0.9027786
## 2 0.8680464 0.8920702 0.9509686 0.8391823
## 3 0.9277097 0.9310306 0.9721062 0.9044418
##
## Clustering vector:
## 10028 10290 10363 10381 10464 10568 10956 11425 11669 11837 11999 12414 12575
##      3      2      2      3      3      3      2      3      3      1      3      3      3
## 13139 13480 13987 14000 14276 14525 14566 14602 15077 15246 15646 15661 16021
##      3      3      3      2      2      3      3      1      2      3      1      3      1
## 16319 16394 1674 16940 17085 17166 17212 17344 17701 18005 18156 18435 1911
##      1      1      3      2      1      2      2      1      2      3      3      1      3
## 19176 19435 19644 19765 20011 20057 20078 20489 21058 21297 21323 21418 21557
##      1      3      1      2      1      1      3      2      2      1      3      3      3
## 21575 21673 22137 22904 22930 23041 23062 23378 23664 23735 23974 2417 24185
##      3      3      1      1      2      1      3      2      2      2      2      3      2
## 24350 24826 25090 25453 25561 2572 26028 26091 26252 26290 26351 26513 26845
##      3      2      1      3      2      1      1      3      2      1      3      2      3
## 26989 2702 27122 27243 27577 27810 28378 28787 28886 29530 29685 2983 29846
##      2      3      3      2      2      2      2      1      3      1      3      2      1
## 29878 30191 30547 30728 30768 30782 30913 31381 31869 31872 32311 32397 33397
##      3      1      2      3      2      1      3      3      3      3      1      3      3
## 33579 33843 33933 34018 34987 35177 35314 35652 35771 35844 35998 36042 36181
##      3      2      3      3      3      3      2      3      3      2      3      2      1
## 36284 36519 36781 36792 37146 37628 3849 39095 39190 39541 39600 40120 40177
##      2      1      3      1      1      2      2      3      3      1      3      1      1

```

```
## 40200 40435 40610 40755 40817 4104 41290 41510 42079 42240 4226 42269 42356
##      3      2      2      2      3      3      3      1      3      3      1      2      2
## 4266 42782 42925 44027 44279 44319 44327 44420 44826 44902 45207 45462 45617
##      3      1      2      2      1      1      2      1      2      2      1      1      3
## 4615 46354 46568 46609 46635 46646 46714 470 47045 47244 47421 478 48067
##      3      1      1      3      3      1      2      3      3      2      1      2      2
## 48251 48360 4842 48576 48997 49084 49328 49553 49630 5001 5808 6533 6954
##      2      1      2      2      3      3      1      1      1      2      2      1      1
## 7438 7601 8061 8115 8117 8684 8824 8843 890 9048 9067 9202 9231
##      3      3      1      3      1      3      2      3      3      3      3      1      3
## 9253 9260 9727 9843 9882
##      2      2      1      1      2
```

```
##
## Within cluster sum of squares by cluster:
## [1] 63.67135 66.88810 88.43162
## (between_SS / total_SS = 17.4 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

```
fviz_nbclust(as.matrix(m),kmeans, method = "wss", k.max = 20)
```



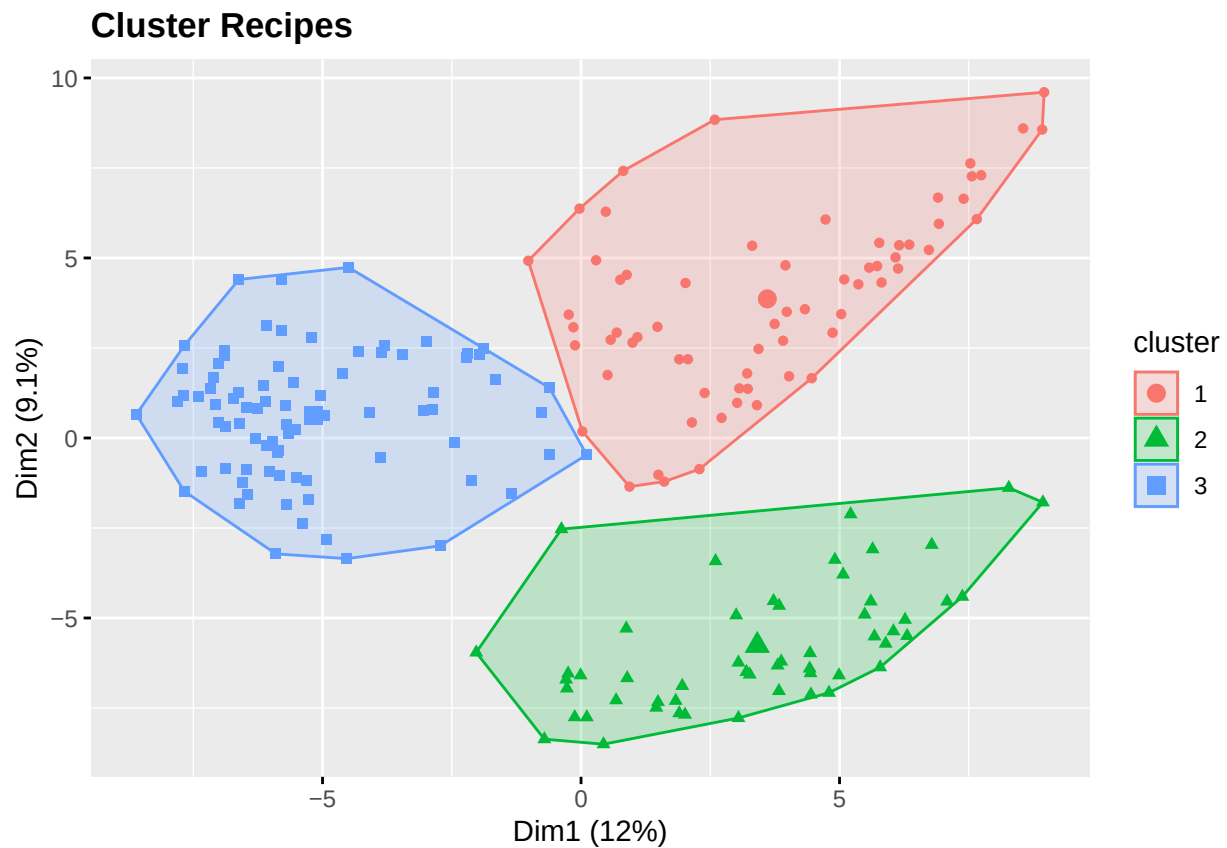
> Base on the elbow method, I would say that 3 clusters should be ideal for this data. This is because the total within sum of square for 2-3 is bigger than the total within sum of square for 3-4. Therefore, 3 clusters would be the optimal number.

Use your chosen number of clusters to cluster the recipes. Use a high `nstart` to ensure a good quality solution.

```
km <- kmeans(as.matrix(m), centers = 3, nstart = 50)
```

Use `fviz_cluster` to visualize the clusters. What patterns do you see?

```
fviz_cluster(km, data = as.matrix(m), geom = "point") +  
  labs(title = "Cluster Recipes") +  
  theme(plot.title = element_text(face = "bold"))
```



> Using the “`fviz_cluster`”, I can see a clear distinction between the 3 cluster. Furthermore, the boundaries of the cluster are bounded by the furthest ingredient/cuisine from the centroid of the cluster.

3. Investigate Clusters

Merge the clusters back into the recipe dataset so that each recipe has a new variable with the cluster ID.

```
x <- tibble(recipe_id = names(km$cluster),
            cluster = as.character(km$cluster))
recipe_id_cluster <- left_join(recipes, x, by = "recipe_id")
```

What are the top 5 ingredients and cuisines in each cluster?

```
recipe_id_cluster %>%
  count(ingredient, cluster) %>%
  group_by(cluster) %>%
  arrange(cluster, desc(n)) %>%
  top_n(5)
```

```
## Selecting by n
```

```
## # A tibble: 17 x 3
## # Groups:   cluster [3]
##   ingredient      cluster      n
##   <chr>          <chr>    <int>
## 1 salt           1         66
## 2 sugar          1         57
## 3 all-purpose flour 1         30
## 4 water          1         19
## 5 unsalted butter 1         14
## 6 sugar          2         53
## 7 water          2         13
## 8 garlic         2         11
## 9 butter         2          9
## 10 soy sauce     2          9
## 11 vegetable oil  2          9
## 12 salt          3         81
## 13 olive oil     3         23
## 14 onions        3         21
## 15 garlic cloves 3         19
## 16 pepper        3         19
## 17 water         3         19
```

For the 1st cluster, the top 5 ingredients are salt, olive oil, onions, garlic cloves, pepper. For the 2nd cluster, the top 5 ingredients are sugar, water, garlic, butter, soy sauce. For the 3rd cluster, the top 5 ingredients are salt, sugar, all-purpose flour, water, unsalted butter

```
recipe_id_cluster %>%
  count(cuisine, cluster) %>%
  group_by(cluster) %>%
  arrange(cluster, desc(n)) %>%
  top_n(5)
```

```
## Selecting by n
```

```
## # A tibble: 15 x 3
## # Groups:   cluster [3]
##   cuisine      cluster      n
##   <chr>        <chr>    <int>
## 1 southern_us    1        148
## 2 french         1         84
## 3 italian        1         69
## 4 chinese        1         67
## 5 british        1         59
## 6 chinese        2         87
## 7 french         2         66
## 8 southern_us    2         65
## 9 vietnamese     2         64
## 10 italian       2         62
## 11 mexican       3        219
## 12 italian       3        169
## 13 indian        3        125
## 14 french        3         93
## 15 cajun_creole  3         69
```

For the 1st cluster, the top 5 cuisine are Mexican, Italian, Indian, french, Cajun_reole. For the 2nd cluster, the top 5 cuisine are Chinese, French, Southern US, Vietnamese, Italian. For the 3rd cluster, the top 5 cuisine are Southern US, French, Italian, Chinese British

4. Conclusion

What patterns did the cluster analysis and PCA from `fviz_cluster` reveal? > **The cluster analysis suggest a clear distinction of clusters such that the boundaries between clusters do not overlap each other. The analysis justify that ingredients/cuisines in cluster 1 has a better grouping than the other analysis. This is because cluster 1 has a higher density of data, while cluster 2 & 3's data spreads out evenly.**

What further cleaning or processing of the data do you think is necessary? > **Firstly, there are too many adjectives for ingredients. For example 'skim milk', 'whole milk' and '1% low fat milk' can be categorize as 'milk'. By providing adjectives, the algorithm will treat all of these ingredients as separate products, which decreases the accuracy of the data. Secondly, this data includes 1 regional cuisine (Southern US) and the rest are nation-based cuisine. I would argue this is an inconsistent categorization of data that would underestimate the data.**

What other visualizations or analyses could you perform explain the patterns that you see in the data? > **There are 3 other visualizations to explain the patterns. Firstly, I would recommend to use PCA Biplot to distinguish the uniqueness of the ingredients/cuisine. I would expect that the further away from the center, it means that an ingredients will be more unique to a specific cuisine. Secondly, I would use a network analysis to test out if there is any relationship between cuisines based on the ingredients given. I would expect common ingredients such as Milk and Sugar to have a higher connection compare to other ingredients. Lastly, I would attempt to make use of a scatterplot with a regression line between 2 different cuisines' ingredients and analyse how similar they are. It helps to group similar cuisines together and creates a cluster with a higher similarities.**